

Reviewer Pack — Minimal Brauer Group Order vs Boolean Circuit Threshold for ...

Entry 10d1747ba9d0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 07:05:41 UTC

Minimal Brauer Group Order vs Boolean Circuit Threshold for Modular Functions

Entry ID: 10d1747ba9d0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 07:05:41 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For every modular function f represented by a circuit C with n inputs and size s , the order of the minimal Brauer group over the function field of f is upper-bounded by $O(s \log^2(n))$.’, ‘There exists a constant c such that for all circuits C representing modular functions, the order of the minimal Brauer group over the function field of f is at least $c \cdot s \log(n)$.’]

Rationale (proposer’s reasoning):

[‘Algebraic K-Theory has been used to study properties of fields in various contexts but rarely in connection with circuit complexity. The Brauer group measures non-triviality of vector bundles over a field, which could capture essential structural information about the complexity of modular functions.’, ‘The conjecture bridges algebraic topology and boolean circuit theory by suggesting that the topological structure encoded in the Brauer group can provide insights into the complexity of computing modular functions.’]

Taxonomy category: ALGEBRAIC_K_THEORY Boolean_Circuit_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 735fff73a1d97330

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal Brauer group order and the circuit size exceeds 0.7 for at least 30 out of 40 seeds, with a mean absolute difference between the observed and expected Brauer group orders less than 10%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Minimal Brauer Group Order" AND "Boolean Circuit Complexity" AND "algebraic K-theory" - "modular function" AND "circuit size" AND "Brauer group order" - "Boolean circuit threshold" AND "modular function field" AND "algebraic K-theory"

Top relevant hits considered: - [s2:7a655050139332ecea067d288fda1992da5383c8] Optogenetic Acidification of Synaptic Vesicles and Lysosomes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def generate_primes(n):
    primes = []
```

```

for num in range(2, n):
    if is_prime(num):
        primes.append(num)
return primes

def generate_circuit(n):
    inputs = random.randint(1, n)
    gates = random.randint(inputs, 5 * inputs)
    circuit = [(random.choice(['AND', 'OR']), [random.randint(0, inputs - 1), random.randint(0, inputs - 1)])]
    return circuit

def compute_function_field(circuit):
    # Simplified function field computation
    return len(circuit)

def compute_brauer_group_order(function_field_size):
    # Simplified Brauer group order computation
    return function_field_size ** 2

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_brauer_group_order = 0
    total_circuit_size = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):
            circuit = generate_circuit(n)
            function_field_size = compute_function_field(circuit)
            brauer_group_order = compute_brauer_group_order(function_field_size)
            total_brauer_group_order += brauer_group_order
            total_circuit_size += len(circuit)
            instances_tested += 1

    mean_brauer_group_order = total_brauer_group_order / instances_tested
    mean_circuit_size = total_circuit_size / instances_tested
    correlation_coefficient = (instances_tested * total_brauer_group_order * total_circuit_size -
                              sum(b * c for b, c in zip([brauer_group_order for _ in range(instances_tested)],
                                                         [total_circuit_size for _ in range(instances_tested)]))) / ((instances_tested * total_brauer_group_order * total_circuit_size -
                                                                                                         (instances_tested * total_brauer_group_order ** 2 -
                                                                                                         instances_tested * total_circuit_size ** 2)))

    conjecture_holds = correlation_coefficient > 0.7
    counterexample = "" if conjecture_holds else "correlation_coefficient < 0.7"

    return {
        "metric_name": "Pearson Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or generate_primes(30)

```

```

results = []

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, **result}}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_638ef142.py", line 101, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_638ef142.py", line 80, in run_trial
    sum(b * c for b, c in zip([brauer_group_order for _ in range(instances_tested)], [circuit_size for _ in range(
                                                                    ~~~~~^~~~~~
NameError: name 'circuit_size' is not defined. Did you mean: 'mean_circuit_size'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing data necessary to evaluate the conjecture. | next: Debug the test code to fix the NameError and rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14779
2	preregistration	ollama_remote	glm4:latest	0	0	9506
3	novelty	ollama_remote	glm4:latest	0	0	8767
4	novelty	ollama_remote	glm4:latest	0	0	9841
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13861
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10673
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10321
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11741
9	judge	ollama_remote	glm4:latest	0	0	20645

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110133 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/10d1747ba9d0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/10d1747ba9d0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/10d1747ba9d0.tar.gz` (if generated)